DAIFEND
SECURING THE AI RUNTIME

PLATFORM GUIDE

FEATURES & HOW TO USE · MAY 2026 · V1.1

Daifend platform guide

Daifend is enterprise AI security infrastructure: **memory integrity** (embedding drift, outliers, prompt-injection signals), **agent runtime policy**, **RAG / threat analysis**, **self-healing orchestration**, **real-time telemetry**, and a **Next.js** operator console. Backends are structured for multi-tenant operation with an **API gateway**, optional **OPA**, **OpenTelemetry**, **Redis** quotas, **PostgreSQL**, **ClickHouse**, **NATS**, **Kafka/Redpanda**, and **gRPC** on the memory path.

1. Feature catalog

WEB CONSOLE (@DAIFEND/WEB)

- **Dashboard** — trust timeline, semantic drift matrix, threat ticker, runtime posture, self-healing feed, metrics series.
- **Modules** — AI Memory Security, Agent Runtime, Threat Intel, Incidents, Self-Healing, Research Lab, Live Simulation, Architecture, API Integrations, Settings.
- **Telemetry client** — Socket.IO to telemetry service; `demo` vs `live/enterprise` ingest modes.
- **Strict mode** — `NEXT_PUBLIC_TELEMETRY_STRICT=true` disables offline UI "fallback" simulation when the socket is down.
- **Security headers** — CSP and related headers via Next.js config (tune `connect-src` for your API hosts).
- **Prisma** — optional app DB access when `DATABASE_URL` is set (e.g. health routes).

API GATEWAY

- **Authentication** — `Bearer` JWT (HS256 from auth-service or OIDC via `OIDC_JWKS_URL`); dev `X-Internal-Token`.
- **Tenant binding** — `X-Tenant-Id` must align with JWT `tenant_id` (except internal token path).
- **RBAC** — route-level permission checks (`memory:*`, `engine:invoke`, `telemetry:read`, etc.).
- **OPA** — optional `OPA_URL` POST to `/v1/data/daifend/gateway/allow` (see `infrastructure/opa/policies/daifend/gateway.rego`).
- **Rate limits** — SlowAPI + Redis (`REDIS_URL`); per-tenant API minute quota + optional memory-scan hourly quota (`daifend_core.quota`).
- **Proxy & resilience** — HTTPX upstream with retries; security headers on responses; optional Sentry (`SENTRY_DSN`).
- **gRPC** — memory analyze can use `MEMORY_GRPC_TARGET` when configured.

AUTH SERVICE

- **POST /v1/oauth/token** — `client_credentials` (access + refresh), `refresh_token` (rotation), `api_key` (exchange stored key for JWT).
- **JWT claims** — `scope` and `permissions` for gateway RBAC; defaults configurable via `DAIFEND_DEFAULT_PERMISSIONS`.
- **API keys** — SHA-256 hash in `api_keys`; scopes JSON drive token permissions. Requires `DATABASE_URL` on auth-service.
- **Refresh storage** — in-process or `REDIS_URL` for horizontal scale.

MEMORY INTEGRITY ENGINE

- **REST** — `POST /v1/analyze` (embeddings + optional text samples); scan, reports, feed, incidents, rollback APIs.
- **Analysis** — deterministic numpy/sklearn pipeline: drift, clustering, outliers, poisoning probability, retrieval anomaly, prompt-injection heuristics.
- **Explainability** — JSON `explainability` block (summary + weighted factors) on analyze responses for auditors.

- **POST /v1/introspect** — token active flag and claims summary.

- **Qdrant / vectors** — optional connector; health exposes vector reachability.
- **Persistence** — PostgreSQL reports/snapshots; optional ClickHouse/NATS integration per deployment.
- **gRPC** — `daifend.v1.MemoryIntegrity/Analyze` for high-throughput internal calls.

OTHER ENGINES

- **Agent runtime** — tool allowlists, argument safety, optional OPA for agent policy package.
- **Threat engine** — RAG document scan and threat correlation APIs.
- **Self-healing** — workflow orchestration (rollback, containment patterns).
- **Notification** — ingress for alerts (Slack/PagerDuty-style adapters extend here).

TELEMETRY SERVICE

- **Socket.IO** — `telemetry:hello`, `telemetry:batch`; ingest mode in hello payload.
- **Demo mode** — synthetic batches for local UI only (`TELEMETRY_INGEST_MODE=demo`).
- **Enterprise mode** — NATS-only (no RNG telemetry); `TELEMETRY_INGEST_MODE=enterprise`.
- **Sinks** — ClickHouse `daifend.telemetry_events_raw`; Kafka topic `daifend.telemetry.raw`.
- **DLQ** — failed ClickHouse or Kafka publish can be written to `KAFKA_TELEMETRY_DLQ_TOPIC` (default `daifend.telemetry.dlq`).

SHARED PLATFORM & COMPLIANCE (REPO)

- **daifend-core** — SQLAlchemy models, Alembic migrations, sync DB helpers, quota helpers, API key verification, HTTP/Observability utilities.
- **TypeScript packages** — `@daifend/types` (telemetry event unions, SDK types), `@daifend/sdk`, config, security, telemetry helpers.
- **Infrastructure** — Docker Compose, Kubernetes base, Helm starter, Terraform VPC baseline, OPA policies, ClickHouse init, GitOps Argo CD example.
- **CI** — web lint/build, Python tests, Bandit, Trivy FS scan, CycloneDX SBOM (Syft); Dependabot for npm, Actions, pip.
- **Documentation** — `COMPLIANCE.md` operator notes; `docs/API-CONTRACTS.md`, `RUNBOOK.md`, `ONBOARDING.md`, etc.

2. How to use — getting started

2.1 PREREQUISITES

Node 22+ (see repo), Python 3.12+ for services, Docker for full stack. Copy `.env.example` to `.env` at repo root.

2.2 LIGHTWEIGHT: UI + DEMO TELEMETRY

```
npm install
npm run dev
npm run telemetry:demo --workspace=@daifend/web
```

Console: `http://127.0.0.1:3000`. Point `NEXT_PUBLIC_TELEMETRY_URL` at your telemetry port if not 4001.

2.3 FULL STACK: DOCKER COMPOSE

```
docker compose up -d --build
```

Endpoint	Purpose
127.0.0.1:3000	Web console
127.0.0.1:8080	API gateway (/health, /ready)
127.0.0.1:8001	Auth service
127.0.0.1:4001	Telemetry (Socket.IO)
127.0.0.1:16686	Jaeger UI (OTLP HTTP 4318)
127.0.0.1:8181	OPA (/v1/data/...)
127.0.0.1:8123	ClickHouse HTTP
127.0.0.1:19092	Redpanda Kafka API
127.0.0.1:5432	PostgreSQL

Compose wires gateway to Redis, OPA, Postgres (quotas/API keys), engines, and telemetry. Set web `NEXT_PUBLIC_DAIFEND_MODE=live` for Python-backed telemetry in the UI.

2.4 DATABASE MIGRATIONS

```
cd packages/daifend-core && pip install -e .
export DATABASE_URL=postgresql://daifend:daifend@localhost:5432/daifend
alembic upgrade head
```

Use `postgresql+psycopg://` URLs in Python services where async/sync drivers differ; Alembic typically uses sync postgres URL.

2.5 PRISMA (WEB WORKSPACE)

```
export DATABASE_URL=postgresql://daifend:daifend@localhost:5432/daifend
npm run db:generate --workspace=@daifend/web
```

3. Authentication & calling the API

3.1 CLIENT CREDENTIALS (DEVELOPMENT)

```
curl -s -X POST http://127.0.0.1:8080/v1/oauth/token \
  -H "Content-Type: application/json" \
  -d '{"tenantId":"default","grantType":"client_credentials"}'
```

In production, supply `clientSecret` when `DAIFEND_ENV` is not development. Response includes `accessToken` and `refreshToken`.

3.2 API KEY EXCHANGE

```
curl -s -X POST http://127.0.0.1:8001/v1/oauth/token \
  -H "Content-Type: application/json" \
  -d '{"tenantId":"<tenant>","grantType":"api_key","apiKey":"<secret>"}'
```

Auth-service must reach Postgres with a valid `api_keys` row (hashed key, scopes JSON). Token permissions mirror stored scopes.

3.3 GATEWAY REQUEST

```
curl -s http://127.0.0.1:8080/v1/memory-integrity/reports \
  -H "Authorization: Bearer <accessToken>" \
  -H "X-Tenant-Id: default"
```

Internal-only engine routes expect `X-Internal-Token` matching `INTERNAL_SERVICE_TOKEN` when engines enforce it. Gateway can forward that header to upstreams.

3.4 TELEMETRY (SOCKET.IO)

Connect a Socket.IO client to the telemetry service URL. On connect, server emits `telemetry:hello` (`mode`: `demo` | `enterprise` | …). Events arrive on `telemetry:batch`; include `tenantId` on each event for sink isolation.

4. Key environment variables (reference)

`JWT_SECRET`	Shared between auth-service and gateway for HS256 validation.
`INTERNAL_SERVICE_TOKEN`	Service-to-service header; gateway and engines.
`REDIS_URL`	Gateway rate limiter + quota backend (optional fallback to memory).
`DATABASE_URL`	Postgres for auth (API keys), gateway quotas, persistence.
`OPA_URL`	e.g. `http://opa:8181`; empty disables OPA (gateway skips policy call).
`OIDC_JWKS_URL`	Optional external IdP JWT verification at gateway.
`TELEMETRY_INGEST_MODE`	`demo` vs `enterprise` (NATS-only).
`KAFKA_BOOTSTRAP_SERVERS`	Enables Kafka sink + DLQ path for telemetry.
`OTEL_EXPORTER_OTLP_ENDPOINT`	e.g. Jaeger collector for traces.
`NEXT_PUBLIC_API_GATEWAY_URL`	Browser-facing gateway URL for live UI.
`NEXT_PUBLIC_TELEMETRY_STRICT`	`true` — no client-side telemetry fallback.

5. Documentation index (repository)

- `README.md` — layout, quick start, CI overview.
- `docs/API-CONTRACTS.md` — v1 HTTP contracts, telemetry, gRPC index.
- `docs/ENTERPRISE-ARCHITECTURE.md` — principles, streaming, K8s, deployment modes.
- `docs/LOCAL_DEV.md` — ports, env, local Python services.
- `docs/HARDENING.md` — OTel, Redis, OIDC, ClickHouse, Kafka.
- `docs/RUNBOOK.md` — health, failure modes, operations.
- `docs/ONBOARDING.md` — engineer setup, K8s notes.
- `COMPLIANCE.md` — controls summary for operators.
- `docs/GRPC.md` — protos and codegen.

6. Regenerating this PDF

```
npm install
npm run docs:pdf --workspace=@daifend/web
```

Requires Playwright (Chromium). Output: `docs/Daifend-Platform-Guide.pdf` from source `docs/daifend-platform-guide.html`.

Daifend — Platform guide v1.1 (May 2026). Behavior is defined by the source repository; verify routes and env vars against your deployment branch.